

Görev Listesi: Java 27 Öğrenim Müfredatı

Aşama: Specify → Plan → Tasks → Implement dizisinin 3. aşaması (Tasks) **Spesifikasyon:** [SPEC.md](#) ·

Plan: [plan.md](#)

Aşağıdaki her görev en fazla 5 dosyaya dokunuyor ve bağımsız olarak doğrulanabilir. Her görev bir GitHub issue'suna karşılık geliyor.

Tüm 24 görev tamamlandı ve issue'ları kapatıldı.

Çeviri notu. Görev metinleri İngilizce özgünlerinden bırakıldı. Dosya adlarını, modül kimliklerini, JEP numaralarını ve kabul kriterlerini isimlendiriyorlar; bunların İngilizce kaynakla birebir eşleşmesi gerekiyor. Çevrilmiş olanlar bölüm başlıkları ve alan etiketleri.

scaffold — iskelet

✓ T01 — Repository scaffolding and verification script · #1

- Kabul: `scripts/verify-examples.sh` walks `modules/**/examples/*.java` and `solutions/**/*.*java`, runs each with JDK 27, honours the `// EXPECT: compile-error` and `// EXPECT: preview` headers, fails on any unexpected non-zero exit, and refuses to run if `java -version` is not 27. Root `README.md` stub states the JDK requirement and how to run an example. `.gitignore` excludes `*.class` and editor cruft.
- Doğrulama: `./scripts/verify-examples.sh` exits 0 on the empty tree; planting a deliberately broken `.java` file makes it exit non-zero with a readable message; removing it restores exit 0.
- Dosyalar: `scripts/verify-examples.sh`, `README.md`, `.gitignore`

template — format referansı

✓ T02 — Module 01 getting-started, the reference implementation · #2

- Kabul: Covers JDK 27 setup, the JVM/JRE/JDK distinction, compact source files (JEP 512, final in 25), `IO.println`, and what compiling actually does. README has all seven template sections. At least two runnable examples. Homework brief with acceptance criteria. Reference solution under `solutions/01-getting-started/`. This module's structure is the template every later module copies.
- Doğrulama: `./scripts/verify-examples.sh modules/01-getting-started` exits 0; solution runs clean; README section headings match the spec template exactly.
- Dosyalar: `modules/01-getting-started/README.md`, `modules/01-getting-started/examples/*.java`, `modules/01-getting-started/homework/README.md`, `solutions/01-getting-started/Solution.java`

- Kapi: T03 başlamadan önce formatın insan tarafından incelenmesi.

core-language — çekirdek dil

✓ T03 — Module 02 variables-and-types · #3

- Kabul: Primitives vs. references, `var` and where it helps or hurts, autoboxing traps including the Integer cache, String immutability, text blocks. Coming-from-another-language section contrasts Python/JS dynamic typing.
- Doğrulama: `./scripts/verify-examples.sh modules/02-variables-and-types` exits 0.
- Dosyalar: `modules/02-variables-and-types/**`, `solutions/02-variables-and-types/**`

✓ T04 — Module 03 control-flow · #4

- Kabul: Operators, `if / else`, switch expressions with arrow form and `yield`, all loop forms, enhanced `for`, labelled break. Contrasts switch expressions against the old fall-through statement form.
- Doğrulama: `./scripts/verify-examples.sh modules/03-control-flow` exits 0.
- Dosyalar: `modules/03-control-flow/**`, `solutions/03-control-flow/**`

✓ T05 — Module 04 methods · #5

- Kabul: Declaration, overloading and its resolution rules, varargs, static vs. instance, and an explicit treatment of Java being pass-by-value including for references, which is the single most misunderstood point for newcomers.
- Doğrulama: `./scripts/verify-examples.sh modules/04-methods` exits 0.
- Dosyalar: `modules/04-methods/**`, `solutions/04-methods/**`

✓ T06 — Module 05 classes-and-objects · #6

- Kabul: Classes, fields, constructors, `this`, encapsulation, static members, flexible constructor bodies (JEP 513, final in 25). Graduates the learner from compact source files to the classic `public class` plus `public static void main(String[] args)` and explains why every real codebase uses it.
- Doğrulama: `./scripts/verify-examples.sh modules/05-classes-and-objects` exits 0.
- Dosyalar: `modules/05-classes-and-objects/**`, `solutions/05-classes-and-objects/**`

✓ T07 — Module 06 inheritance-and-polymorphism · #7

- Kabul: `extends`, `super`, `@Override`, `abstract`, `final`, dynamic dispatch, and the `equals / hashCode / toString` contract with a worked example of getting it wrong.
- Doğrulama: `./scripts/verify-examples.sh modules/06-inheritance-and-polymorphism` exits 0.
- Dosyalar: `modules/06-inheritance-and-polymorphism/**`, `solutions/06-inheritance-and-polymorphism/**`

✓ T08 — Module 07 interfaces · #8

- Kabul: Interfaces vs. abstract classes, default/static/private methods, functional interfaces introduced as a concept ahead of Module 13, and why Java favours composition here.
- Doğrulama: `./scripts/verify-examples.sh modules/07-interfaces` exits 0.
- Dosyalar: `modules/07-interfaces/**`, `solutions/07-interfaces/**`

✓ T09 — Module 08 records-and-enums · #9

- Kabul: Records, compact constructors, what records generate for you, enums carrying behaviour and state, and a decision guide for class vs. record vs. enum.
- Doğrulama: `./scripts/verify-examples.sh modules/08-records-and-enums` exits 0.
- Dosyalar: `modules/08-records-and-enums/**`, `solutions/08-records-and-enums/**`

✓ T10 — Module 09 sealed-and-pattern-matching · #10

- Kabul: Sealed types, `instanceof` patterns, switch pattern matching, record patterns, and exhaustiveness checking. Depends on records from Module 08.
- Doğrulama: `./scripts/verify-examples.sh modules/09-sealed-and-pattern-matching` exits 0.
- Dosyalar: `modules/09-sealed-and-pattern-matching/**`, `solutions/09-sealed-and-pattern-matching/**`

✓ T11 — Module 10 exceptions · #11

- Kabul: Checked vs. unchecked and why Java is alone in this, try/catch/finally, try-with-resources, custom exceptions, and an explicit list of anti-patterns such as swallowing exceptions.
- Doğrulama: `./scripts/verify-examples.sh modules/10-exceptions` exits 0.
- Dosyalar: `modules/10-exceptions/**`, `solutions/10-exceptions/**`

type-system — tip sistemi

✓ T12 — Module 11 generics · #12

- Kabul: Generic classes and methods, bounded types, wildcards with PECS, type erasure and the concrete limitations it imposes.
- Doğrulama: `./scripts/verify-examples.sh modules/11-generics` exits 0.
- Dosyalar: `modules/11-generics/**`, `solutions/11-generics/**`

✓ T13 — Module 12 collections · #13

- Kabul: `List`, `Set`, `Map`, `Deque`, sequenced collections, choosing the right implementation, the equals/hashCode contract applied to hash-based collections, and immutable factory methods.
- Doğrulama: `./scripts/verify-examples.sh modules/12-collections` exits 0.

- Dosyalar: `modules/12-collections/**`, `solutions/12-collections/**`

functional — fonksiyonel

✓ T14 — Module 13 lambdas · #14

- Kabul: Lambda syntax, method references in all four forms, the standard functional interfaces, and variable capture including the effectively-final rule.
- Doğrulama: `./scripts/verify-examples.sh modules/13-lambdas` exits 0.
- Dosyalar: `modules/13-lambdas/**`, `solutions/13-lambdas/**`

✓ T15 — Module 14 streams · #15

- Kabul: Pipeline anatomy, intermediate vs. terminal operations, laziness, collectors, gatherers, and an honest treatment of when parallel streams help and when they hurt. Depends on Module 13.
- Doğrulama: `./scripts/verify-examples.sh modules/14-streams` exits 0.
- Dosyalar: `modules/14-streams/**`, `solutions/14-streams/**`

✓ T16 — Module 15 optional · #16

- Kabul: `Optional` as a return type, the idiomatic operations, and the anti-patterns that make it worse than plain null such as calling `get` unguarded or using it as a field.
- Doğrulama: `./scripts/verify-examples.sh modules/15-optional` exits 0.
- Dosyalar: `modules/15-optional/**`, `solutions/15-optional/**`

platform — platform

✓ T17 — Module 16 files-and-io · #17

- Kabul: `Path` and `Files`, reading and writing text, streaming large files, try-with-resources applied for real, and why the legacy `File` API is avoided.
- Doğrulama: `./scripts/verify-examples.sh modules/16-files-and-io` exits 0; examples clean up any files they create.
- Dosyalar: `modules/16-files-and-io/**`, `solutions/16-files-and-io/**`

✓ T18 — Module 17 standard-library-tour · #18

- Kabul: String methods and formatting, `java.time` in depth enough to avoid the classic date bugs, `Math`, random number generation, and comparators.
- Doğrulama: `./scripts/verify-examples.sh modules/17-standard-library-tour` exits 0; no example depends on the current wall-clock time for its assertions.
- Dosyalar: `modules/17-standard-library-tour/**`, `solutions/17-standard-library-tour/**`

✓ T19 — Module 18 concurrency-basics · #19

- Kabul: Threads, a demonstrable race condition, `synchronized`, `atomics`, `ExecutorService`, and the memory-visibility problem stated plainly.
- Doğrulama: `./scripts/verify-examples.sh modules/18-concurrency-basics` exits 0; examples are deterministic enough not to flake, or state their nondeterminism and still exit 0.
- Dosyalar: `modules/18-concurrency-basics/**`, `solutions/18-concurrency-basics/**`

✓ T20 — Module 19 virtual-threads · #20

- Kabul: Virtual threads (final in 21), why blocking became cheap again, scoped values (JEP 506, final in 25), pinning pitfalls, and a benchmark example contrasting platform and virtual threads.
- Doğrulama: `./scripts/verify-examples.sh modules/19-virtual-threads` exits 0.
- Dosyalar: `modules/19-virtual-threads/**`, `solutions/19-virtual-threads/**`

✓ T21 — Module 20 packages-and-modules · #21

- Kabul: Packages, classpath, the module system at a practical level, module import declarations (JEP 511, final in 25), plus `jar` and `javadoc` usage.
- Doğrulama: `./scripts/verify-examples.sh modules/20-packages-and-modules` exits 0; any multi-file example documents its exact compile command.
- Dosyalar: `modules/20-packages-and-modules/**`, `solutions/20-packages-and-modules/**`

✓ T22 — Module 21 testing-your-code · #22

- Kabul: `assert` with `-ea`, a hand-rolled micro test harness kept under 50 lines, what a real JUnit setup adds and why projects adopt it. Zero third-party dependencies, per the spec boundary.
- Doğrulama: `./scripts/verify-examples.sh modules/21-testing-your-code` exits 0; harness demonstrates both a passing and a failing assertion without failing the overall run.
- Dosyalar: `modules/21-testing-your-code/**`, `solutions/21-testing-your-code/**`

release-literacy — sürüm okuryazarlığı

✓ T23 — Module 22 whats-new · #23

- Kabul: What JDK 26 and 27 actually changed, sourced from `openjdk.org` rather than memory. LTS vs. non-LTS and why 25 matters more than 27 for production. How to read a JEP. Preview and incubator features explained, with structured concurrency (JEP 533, seventh preview) and primitive patterns (JEP 532, fifth preview) demonstrated behind `--enable-preview`, each labelled with its JEP number and preview round.
- Doğrulama: `./scripts/verify-examples.sh modules/22-whats-new` exits 0 including the preview-flagged examples; every version claim in the README carries a link to its `openjdk.org` source.
- Dosyalar: `modules/22-whats-new/**`, `solutions/22-whats-new/**`

integration — bütünleştirme

✅ T24 — Root README and final verification pass · #24

- Kabul: Root `README.md` links all 22 modules in order with a one-line description each, documents JDK 27 setup per platform, and explains how to use the solutions tree. All eight spec success criteria checked off explicitly.
- Doğrulama: `./scripts/verify-examples.sh` exits 0 across the whole repository; every module folder confirmed to contain README, examples, and homework; no non-English content found; no third-party dependency anywhere.
- Dosyalar: `README.md` , `SPEC.md` (success criteria checklist)